

Experiment No. 5: Programs to study Python Functions, Function Arguments, Variable Scope, and Global Keyword.

Aim: To write and execute Python programs to understand functions, different types of function arguments, scope of variables, and the use of the global keyword.

Objectives:

After performing this experiment, the student will be able to:

1. Define and call Python functions.
2. Use different types of function arguments.
3. Understand local and global variables.
4. Use the global keyword to modify global variables inside a function.

Theory:

1. Python Function

A function is a block of code that performs a specific task.

It helps in:

- Code reusability
- Better readability
- Easy debugging

Syntax:

```
def function_name(parameters):  
    statements
```

Example:

```
def add(a, b):  
    return a + b
```

2. Function Arguments

Arguments are values passed to a function.

Types of arguments:

1. Positional Arguments
2. Keyword Arguments
3. Default Arguments
4. Variable Length Arguments

3. Variable Scope

Scope defines where a variable can be accessed.

- Local Variable → defined inside a function
- Global Variable → defined outside all functions

4. Global Keyword

The global keyword is used to:

- Access and modify a global variable inside a function.

Example:

```
x = 10
def change():
    global x
    x = 20
```

Program 1: Simple Function, To define and call a simple function.

```
def greet():
    print("Welcome to Python Programming")

greet()
```

Program 2: Function with Arguments

```
def add(a, b):    # a and b are parameters (inputs to the function).
    c = a + b
    print("Sum =", c)

add(10, 20)      # Calls the function with arguments 10 and 20.
```

with User Input

```
def add(a, b):
    c = a + b
    print("Sum =", c)

# Take input from user
x = int(input("Enter first number: "))
y = int(input("Enter second number: "))

# Call function with user input values
add(x, y)
```

Program 3: Positional and Keyword Arguments

```
def display(name, age):      # function with two parameters: name and age
    print("Name:", name)
    print("Age:", age)

display("Alice", 20)         # Positional arguments: "Alice" → name, 20 → age

display(age=22, name="Bob")  # Order does NOT matter in keyword arguments.
```

Program 4: Default Arguments

```
# Defines a function with a default value for parameter country
def show(country="India"):

    print("Country:", country)

show()    # No argument is passed, So default value "India" is used.

show("USA") # Argument "USA" is passed, It replaces the default value "India".
```

Program 5: Variable Length Arguments

*numbers mean the function can accept any number of arguments. All arguments are stored in a tuple called numbers.

```
def total(*numbers):
    sum = 0
    for n in numbers:
        sum = sum + n
    print("Total =", sum)

total(10, 20)
total(10, 20, 30, 40)
```

with User Input

```
def total(*numbers):
```

```
    sum = 0
```

```
    for n in numbers:
```

```
        sum = sum + n
```

```
    print("Total =", sum)
```

```
# Take how many numbers from user
```

```
n = int(input("How many numbers do you want to add? "))
```

```
# Creates an empty list named values. This will store the numbers entered by the user.
```

```
values = []
```

```
for i in range(n):
```

```
    num = int(input(f'Enter number {i+1}: ')) # Takes each number from the user.
```

```
    values.append(num) # Adds the entered number to the list 'values'.
```

```
# Calls the function total. *values unpacks the list and passes each element as a  
separate argument. Example: if values = [10, 20, 30] then total(*values) becomes:  
total(10, 20, 30)
```

```
total(*values)
```

Program 6: Local Variable

```
def myfunc():
```

```
    x = 10
```

```
# Local variable, x is created inside the function. It exists only inside myfunc().
```

```
    print("Inside function x =", x)
```

```
myfunc()
```

```
# print(x) # If executed, it will give an error. Error: x is not defined outside
```

Program 7: Global Variable

```
# Global variables can be accessed both inside and outside the function.
```

```
x = 5 # Global variable, x is declared outside the function.
```

```
def show():
```

```
    print("Inside function x =", x)
```

```
show()
```

```
print("Outside function x =", x)
```

Program 8: Using Global Keyword

The global keyword is used to modify a global variable inside a function.

`x = 10` *# Declares a global variable x.*

`def change():`

`global x`

`x = 50` *# Changes the value of global variable x from 10 to 50.*

`print("Inside function x =", x)`

`change()` *# Calls the function and modifies the global variable.*

`print("Outside function x =", x)` *# Prints the updated global value*

Program 9: Write a program to demonstrate the use of local variable, global variable and global keyword.

`x = 10` *# Global variable*

`def change():`

`y = 5` *# Local variable*

`global x` *# Access global variable*

`x = x + y`

`print("Inside function:")`

`print("Local variable y =", y)`

`print("Global variable x =", x)`

`print("Before function call, x =", x)`

`change()`

`print("After function call, x =", x)`

Problem Statement 1: Write a function to calculate square of a number.

```
def square(n):  
    return n * n
```

```
num = int(input("Enter a number: "))  
result = square(num)  
print("Square =", result)
```

Problem statement 2: Write a function to display employee details using keyword arguments.

```
def employee(emp_id, name, salary):  
    print("Employee ID:", emp_id)  
    print("Name:", name)  
    print("Salary:", salary)
```

```
employee(name="Rahul", salary=45000, emp_id=101)
```

Problem Statement 3: Write a function to calculate simple interest where the rate of interest is defaulted to 5%.

```
def simple_interest(p, t, r=5):  
    si = (p * t * r) / 100  
    print("Simple Interest =", si)
```

```
simple_interest(1000, 2)    # uses default rate
```

```
simple_interest(1000, 2, 8) # uses given rate
```

Problem Statement 4: Write a function to find the maximum among any number of values using variable-length arguments.

```
def find_max(*numbers):  
    max_val = numbers[0]  
    for n in numbers:  
        if n > max_val:  
            max_val = n  
    print("Maximum value =", max_val)
```

```
find_max(10, 25, 5)
```

```
find_max(100, 45, 78, 23, 9)
```

Problem Statement 5: Write functions for addition, subtraction, multiplication and division.

```
def add(a, b):  
    return a + b  
def sub(a, b):  
    return a - b  
def mul(a, b):  
    return a * b  
def div(a, b):  
    return a / b
```

```
x = int(input("Enter first number: "))  
y = int(input("Enter second number: "))
```

```
print("Addition =", add(x, y))  
print("Subtraction =", sub(x, y))  
print("Multiplication =", mul(x, y))  
print("Division =", div(x, y))
```

Problem Statement 6: Write a function to find factorial of a number.

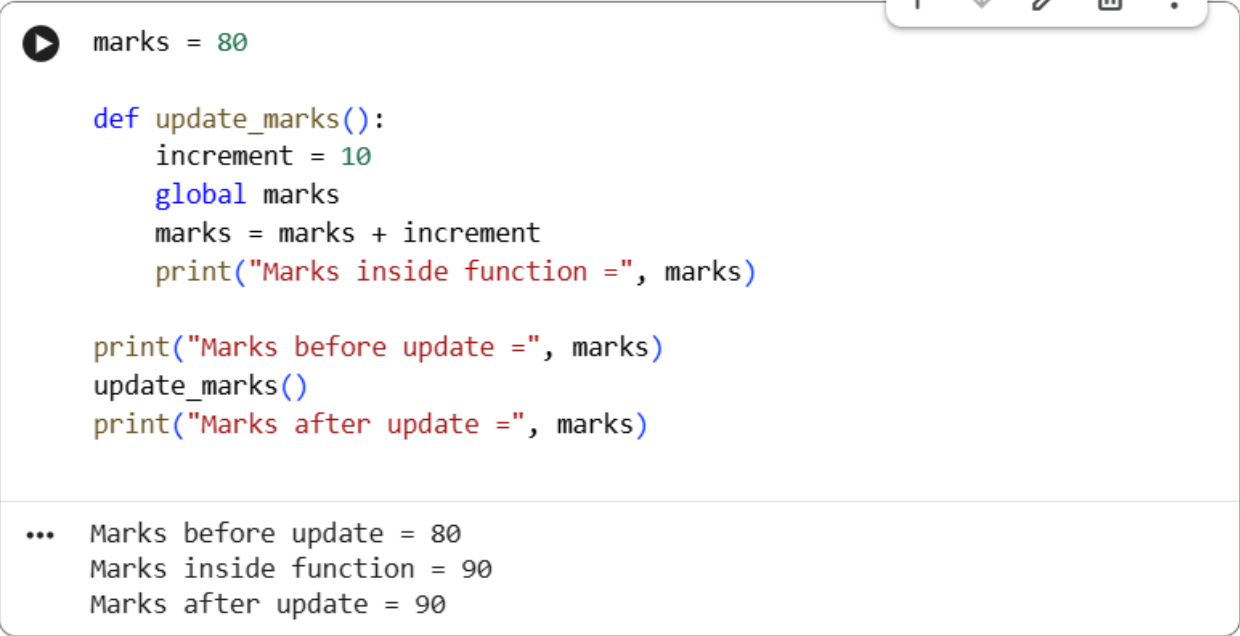
```
def factorial(n):  
    fact = 1  
    for i in range(1, n+1):  
        fact = fact * i  
    return fact
```

```
num = int(input("Enter a number: "))  
print("Factorial =", factorial(num))
```

Problem Statement 7: Write a program to update bank balance using global variable and local variable.

```
balance = 1000 # Global variable  
def deposit(amount):  
    global balance  
    bonus = 50 # Local variable  
    balance = balance + amount + bonus  
    print("Deposited:", amount)  
    print("Bonus added:", bonus)  
  
print("Initial balance =", balance)  
deposit(500)  
print("Final balance =", balance)
```

Problem Statement 8: Write a program to update student marks using global keyword.

A screenshot of a Python code editor window. The editor has a toolbar at the top right with icons for undo, redo, search, and other functions. The code is written in a light blue font on a white background. It defines a function 'update_marks()' that uses the 'global' keyword to modify a variable 'marks' from 80 to 90. The code includes print statements to show the value of 'marks' before and after the function call.

```
marks = 80

def update_marks():
    increment = 10
    global marks
    marks = marks + increment
    print("Marks inside function =", marks)

print("Marks before update =", marks)
update_marks()
print("Marks after update =", marks)
```

```
... Marks before update = 80
    Marks inside function = 90
    Marks after update = 90
```


Experiment 6: Programs to study Python Recursion, Modules, Python Package and Main Function

Aim:

To write and execute Python programs to understand the concepts of recursion, create and use modules and packages, and comprehend the use of the `__main__` function.

Objectives:

After performing this experiment, the student will be able to:

1. Understand and implement recursive functions to solve problems.
2. Create and import custom modules to organize code.
3. Structure code into packages for larger projects.
4. Understand the purpose and use of the `if __name__ == "__main__":` construct.

Theory:

1. Recursion

Recursion is a programming technique where a function calls itself to solve a smaller instance of the same problem. It is an alternative to iteration (loops) for certain types of problems, often leading to cleaner and more readable code.

A recursive function must have two essential components:

- **Base Case:** A condition that stops the recursion. Without a base case, the function would call itself infinitely, leading to a stack overflow error.
- **Recursive Case:** The part of the function where it calls itself, moving towards the base case.

Example: Calculating the factorial of a number ($n!$) is a classic example of recursion.

- $5! = 5 * 4 * 3 * 2 * 1$
- This can be defined as: `factorial(n) = n * factorial(n-1)`
- **Base Case:** `factorial(1) = 1`

2. Modules

A module is a file containing Python definitions and statements. It allows you to logically organize your Python code. As your program grows, you can split it into multiple files (modules) for better maintainability and reusability.

- You can create your own modules (e.g., `mymodule.py`).
- You can use built-in modules (e.g., `math`, `random`).
- You can import a module's functions and variables into your current program using the `import` statement.

Example: `import math` allows you to use functions like `math.sqrt()`.

3. Packages

A package is a way of organizing related modules into a single directory hierarchy. Think of it as a folder containing multiple module files (`.py` files) and a special `__init__.py` file. This `__init__.py` file is required to let Python know that the directory should be treated as a package.

Structure Example:

```
text
mypackage/
  __init__.py
  module1.py
  module2.py
```

You can then import from the package using statements like `import mypackage.module1`.

4. The `__main__` Function / Guard

In Python, the `if __name__ == "__main__":` block is a common and important construct. It acts as a entry point for the program, but its primary purpose is to allow a Python file to be used both as a reusable module and as a standalone script.

- When you run a Python script directly (e.g., `python myprogram.py`), the special built-in variable `__name__` is set to `"__main__"`.
- When you import that same file as a module into another script, `__name__` is set to the module's name (e.g., `myprogram`).

This block allows you to put code that should only run when the file is executed directly (like test code or the main program logic) and prevent it from running when the file is imported elsewhere.

Program 1: Recursion - Factorial of a Number

```
def factorial(n):
    """Calculates the factorial of a number recursively."""
    # Base case: factorial of 1 or 0 is 1
    if n == 1 or n == 0:
        return 1
    # Recursive case:  $n! = n * (n-1)!$ 
    else:
        return n * factorial(n - 1)

# Get input from user
num = int(input("Enter a number to find its factorial: "))

# Check for negative numbers
if num < 0:
    print("Factorial is not defined for negative numbers.")
else:
    result = factorial(num)
    print(f"The factorial of {num} is: {result}")
```

Program 3: Using Built-in Module

```
import math

num = float(input("Enter a number: "))

print("Square root:", math.sqrt(num))
print("Power:", math.pow(num, 3))
```

Program 4: Create and Use User-Defined Module

Create file mymodule.py:

```
def square(n):
    return n * n
```

Main file:

```
import mymodule

num = int(input("Enter a number: "))
print("Square =", mymodule.square(num))
```

Program 5: Demonstration of Main Function

```
def display():  
    print("This is a function")  
  
if __name__ == "__main__":  
    print("File executed directly")  
    display()
```

Program 6: Using Package

Assume folder structure:

```
mypackage/  
    __init__.py  
    operations.py  
operations.py:
```

```
def add(a, b):  
    return a + b
```

Main file:

```
from mypackage import operations  
  
print("Sum =", operations.add(10, 20))
```

In Colab, we create a .py file inside the notebook environment.

Step 1: Create a Module File

In a new cell, write:

```
%%writefile mymodule.py
```

```
def add(a, b):  
    return a + b
```

```
def sub(a, b):  
    return a - b
```

Explanation:

- %%writefile is a Colab magic command.
- It creates a Python file in the current working directory.

After running this cell, Colab creates:

mymodule.py

Step 2: Import the Module

In the next cell:

```
import mymodule
```

```
print("Addition:", mymodule.add(10, 5))  
print("Subtraction:", mymodule.sub(10, 5))
```

✓ Module works successfully.

Step 3: If You Modify the Module

If you edit mymodule.py, reload it:

```
import importlib
```

```
importlib.reload(mymodule)
```

This is important in Colab because it caches modules.

🔗 PART 2 — Creating a Package in Google Colab

A package is a folder containing:

- One or more .py files
 - A special file `__init__.py`
-

Step 1: Create Folder (Package)

```
import os
```

```
os.mkdir("mypackage")
```

Step 2: Create init.py

```
%%writefile mypackage/__init__.py
```

```
# This file makes the folder a package
```

Step 3: Create Module Inside Package

```
%%writefile mypackage/operations.py
```

```
def multiply(a, b):  
    return a * b
```

```
def divide(a, b):  
    return a / b
```

Step 4: Import From Package

```
from mypackage import operations
```

```
print("Multiplication:", operations.multiply(4, 5))  
print("Division:", operations.divide(20, 4))
```

Problem Statements

1. Write a recursive function to find sum of first n natural number.

#Write a recursive function to find sum of first n natural numbers.

```
def sum_natural_numbers(n):  
    if n == 0:  
        return 0  
    else:  
        return n + sum_natural_numbers(n - 1)  
sum_natural_numbers(10)
```

2. Write a recursive function to calculate power (x^n).

#Write a recursive function to calculate power (x^n).

```
def power(x, n):  
    if n == 0:  
        return 1  
    elif n > 0:  
        return x * power(x, n - 1)  
    else:  
        return 1 / x * power(x, n + 1)  
power(4,3)
```

3. Create a user-defined module to perform arithmetic operations.

```
#Create a user-defined module to perform arithmetic operations.
%%writefile arithmetic.py
def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

def multiply(a, b):
    return a * b

def divide(a, b):
    if b == 0:
        return "Cannot divide by zero"
    return a / b
```

... Writing arithmetic.py

```
import arithmetic
print(arithmetic.add(5, 3))
print(arithmetic.subtract(5, 3))
print(arithmetic.multiply(5, 3))
print(arithmetic.divide(5, 3))
```

```
8
2
15
1.6666666666666667
```

4. Create a package containing two modules: one for area calculations and one for volume calculations.

```
import os
os.mkdir("new_package")
```

```
%%writefile new_package/__init__.py
```

```
... Writing new_package/__init__.py
```

```
%%writefile new_package/area.py
def circle(radius):
    return 3.14159 * radius**2
def square(side):
    return side * side
def rect(length, width):
    return length * width
```

```
Overwriting new_package/area.py
```

```
%%writefile new_package/volume.py
def sphere(radius):
    return 1.33 * 3.14159 * radius**3
def cube(side):
    return side * side * side
def cuboid(length, width, height):
    return length * width * height
```

```
Writing new_package/volume.py
```

```
▶ #Create a package containing two modules: one for area calculations and o
from new_package import area, volume
print("Circle:", area.circle(3))
print("Square:", area.square(3))
print("Rect:", area.rect(3,4))
print("Sphere:", volume.sphere(3))
print("Cube:", volume.cube(3))
print("Cuboid:", volume.cuboid(3,4,5))
```

```
... Circle: 28.27431
Square: 9
Rect: 12
Sphere: 112.81449690000001
Cube: 27
Cuboid: 60
```


5. Demonstrate the use of if `__name__ == "__main__"`.

#Demonstrate the use of if `__name__ == "__main__"`.

```
def add(a, b):
    return a + b

if __name__ == "__main__":
    print("Running Calculator Script Directly")
    num1 = 10
    num2 = 5
    result = add(num1, num2)

    print(f"Test Result: {num1} + {num2} = {result}")
```

6. Compare recursive and iterative factorial implementation.

```
def factorial_recursive(n):
    if n == 0 or n == 1:
        return 1
    return n * factorial_recursive(n - 1)

def factorial_iterative(n):
    result = 1
    for i in range(1, n + 1):
        result *= i
    return result

n = 5
print(f"Recursive Result: {factorial_recursive(n)}")
print(f"Iterative Result: {factorial_iterative(n)}")
```

7. Write a recursive function to reverse a string.

```
def reverse_string(s):
    if len(s) == 0:
        return s
    else:
        return reverse_string(s[1:]) + s[0]

text = input("Enter a string to reverse: ")
print("Reversed string:", reverse_string(text))
```

Experiment 7: Python Exceptions, Exception Handling and Custom Exceptions

Aim:

To study Python exceptions and implement exception handling using try, except, else, and finally blocks, and to create custom exceptions in Python.

Objectives

- To understand exceptions in Python
- To use try and except blocks for error handling
- To handle multiple exceptions in a program
- To use else and finally blocks
- To create and use custom exceptions

Theory

1. Exceptions in Python

An **exception** is an error that occurs during the execution of a program. When an exception occurs, the program stops its normal execution.

Common exceptions in Python include:

- `ZeroDivisionError` – Occurs when dividing by zero
- `ValueError` – Occurs when invalid input is given
- `TypeError` – Occurs when incompatible data types are used
- `IndexError` – Occurs when accessing invalid list index

Example:

```
print(10/0)
```

This produces a **ZeroDivisionError**.

2. Exception Handling

Python provides **exception handling** to manage errors using the try and except blocks.

Syntax:

```
try:
```

```
    # risky code
```

```
except ExceptionType:
```

```
    # handling code
```

Example:

```
try:
    x = int(input("Enter a number: "))
    print(10/x)
except ZeroDivisionError:
    print("Cannot divide by zero")
```

3. Multiple Exceptions

A program may contain multiple exception blocks to handle different errors.

Example:

```
try:
    num = int(input("Enter a number: "))
    result = 10 / num
    print(result)

except ZeroDivisionError:
    print("Division by zero is not allowed")

except ValueError:
    print("Invalid input")
```

4. else and finally Blocks

- **else** runs when no exception occurs
- **finally** always runs whether an exception occurs or not

Example:

```
try:
    num = int(input("Enter number: "))
    print(10/num)

except ZeroDivisionError:
    print("Division by zero")

else:
    print("Operation successful")

finally:
    print("Program execution completed")
```

5. Custom Exceptions

Python allows users to define their own exceptions by creating a class derived from Exception.

Example:

```
class NegativeNumberError(Exception):  
    pass
```

Custom exceptions are useful for handling application-specific errors.

Programs

Program 1: Handling ZeroDivisionError

```
try:  
    a = int(input("Enter numerator: "))  
    b = int(input("Enter denominator: "))  
    result = a / b  
    print("Result:", result)  
  
except ZeroDivisionError:  
    print("Error: Division by zero is not allowed")
```

Program 2: Handling ValueError

```
try:  
    num = int(input("Enter an integer: "))  
    print("You entered:", num)  
  
except ValueError:  
    print("Invalid input. Please enter a number.")
```

Program 3: Handling Multiple Exceptions

```
try:  
    num = int(input("Enter number: "))  
    result = 100 / num  
    print("Result:", result)  
  
except ZeroDivisionError:  
    print("Cannot divide by zero")  
  
except ValueError:  
    print("Invalid input")
```

Program 4: Using finally Block

```
try:
    num = int(input("Enter a number: "))
    print(10 / num)

except ZeroDivisionError:
    print("Division by zero is not allowed")

finally:
    print("Execution completed")
```

Program 5: Using else Block

```
try:
    num = int(input("Enter a number: "))
    result = 50 / num
except ZeroDivisionError:
    print("Division by zero")

else:
    print("Result:", result)
```

Program 6: IndexError Example

```
numbers = [10, 20, 30]

try:
    print(numbers[5])

except IndexError:
    print("Index out of range")
```

Program 7: Custom Exception Example

```
class NegativeNumberError(Exception):
    pass

num = int(input("Enter a positive number: "))
try:
    if num < 0:
        raise NegativeNumberError
    print("You entered:", num)

except NegativeNumberError:
    print("Negative numbers are not allowed")
```

Problem Statement

Problem 1

Write a Python program to:

1. Accept two numbers from the user.
2. Perform division of the numbers.
3. Handle ZeroDivisionError if denominator is zero.
4. Display an appropriate error message.

```
try:
    a = float(input("Enter first number: "))
    b = float(input("Enter second number: "))

    result = a / b
    print("Result:", result)

except ZeroDivisionError:
    print("Cannot divide by zero")
|

... Enter first number: 5
Enter second number: 0
Cannot divide by zero
```

Problem 2

Write a Python program to:

1. Accept a number from the user.
2. Convert it into an integer.
3. Handle ValueError if the user enters invalid input.
4. Display a proper message.

```
try:
    num = int(input("Enter a number: "))
    print("You entered:", num)

except ValueError:
    print("Invalid input. Please enter numeric value.")

... Enter a number: 45te
Invalid input. Please enter numeric value.
```

Problem 3: Student Age Validation

Write a Python program to:

1. Accept the age of a student.
2. If age is less than 0 or greater than 120, raise a custom exception `InvalidAgeError`.
3. Display a message if age is valid.



```
class InvalidAgeError(Exception):
    pass

age = int(input("Enter age: "))

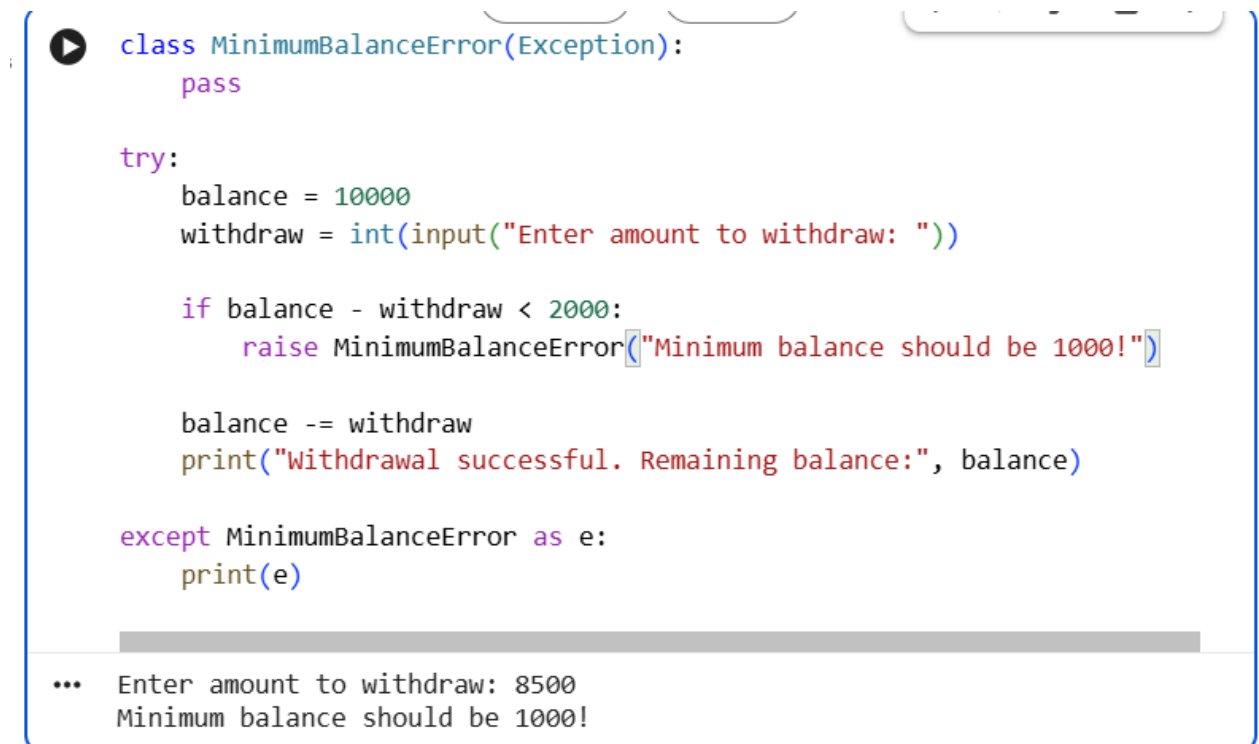
try:
    if age < 0 or age > 120:
        raise InvalidAgeError
    print("Valid Age")

except InvalidAgeError:
    print("Invalid age entered")
```

... Enter age: 128
Invalid age entered

Problem 4: Minimum Balance Exception

Create a bank program where withdrawal should not make balance less than ₹1000. Otherwise raise a custom exception.



```
class MinimumBalanceError(Exception):
    pass

try:
    balance = 10000
    withdraw = int(input("Enter amount to withdraw: "))

    if balance - withdraw < 2000:
        raise MinimumBalanceError("Minimum balance should be 1000!")

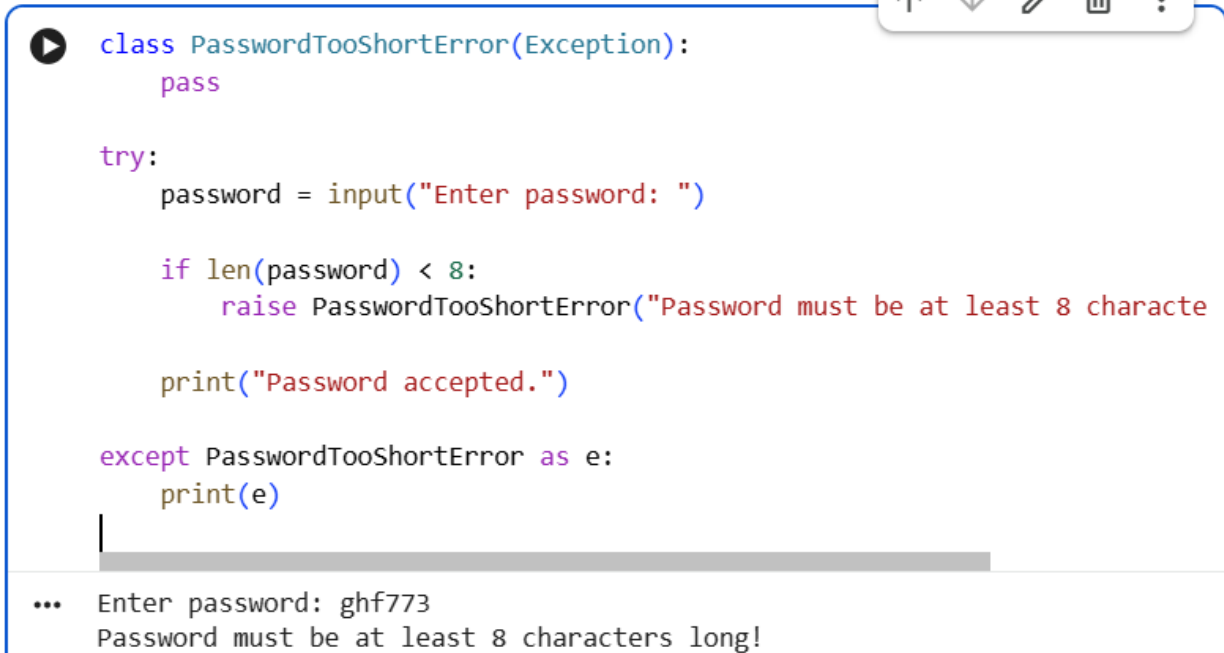
    balance -= withdraw
    print("Withdrawal successful. Remaining balance:", balance)

except MinimumBalanceError as e:
    print(e)
```

... Enter amount to withdraw: 8500
Minimum balance should be 1000!

Problem 5: Password Length Exception

Create a program that checks if password length is at least 8 characters. Otherwise raise a custom exception.



```
class PasswordTooShortError(Exception):  
    pass  
  
try:  
    password = input("Enter password: ")  
  
    if len(password) < 8:  
        raise PasswordTooShortError("Password must be at least 8 characters long")  
  
    print("Password accepted.")  
  
except PasswordTooShortError as e:  
    print(e)
```

... Enter password: ghf773
Password must be at least 8 characters long!

Experiment 8: File Handling – Programs to Perform Various Operations on a File

Aim:

To study file handling in Python and implement programs to create, read, write, append, and perform various operations on files.

Objectives

- To understand file handling concepts in Python
 - To create and open files using different modes
 - To read data from a file
 - To write and append data to a file
 - To perform operations such as counting words and lines in a file
-

Theory

1. File Handling in Python

File handling allows programs to **store and retrieve data permanently** from files. Python provides built-in functions to work with files.

A file must be **opened before performing operations** and **closed after completing the task**.

Basic syntax:

```
file_object = open("filename", "mode")
```

Modes specify the type of operation to perform on the file.

2. File Opening Modes

Common file modes include:

r – Read mode (default mode)

w – Write mode (creates a new file or overwrites existing file)

a – Append mode (adds data to the end of the file)

x – Create mode (creates a new file and gives error if file exists)

Example:

```
f = open("data.txt", "w")
```

3. Writing to a File

The write() function is used to write data into a file.

Example:

```
file = open("data.txt", "w")
file.write("Hello Python")
file.close()
```

4. Reading from a File

Python provides several functions to read file contents.

`read()` – Reads entire file

`readline()` – Reads one line at a time

`readlines()` – Reads all lines and returns a list

Example:

```
file = open("data.txt", "r")
print(file.read())
file.close()
```

5. Appending Data to a File

The append mode allows adding new content without deleting existing data.

Example:

```
file = open("data.txt", "a")
file.write("\nWelcome to File Handling")
file.close()
```

6. Closing a File

After completing operations, the file should be closed using:

```
file.close()
```

Closing a file releases system resources and ensures data is saved properly.

Programs

Program 1: Create and Write Data to a File

```
file = open("sample.txt", "w")
file.write("Python File Handling Example")
file.close()
```

```
print("Data written to file successfully")
```

Program 2: Read Data from a File

```
file = open("sample.txt", "r")
content = file.read()
```

```
print("File Content:")
print(content)
```

```
file.close()
```

Program 3: Append Data to a File

```
file = open("sample.txt", "a")
file.write("\nThis line is added using append mode.")
file.close()
```

```
print("Data appended successfully")
```

Program 4: Read File Line by Line

```
file = open("sample.txt", "r")
```

```
for line in file:
    print(line)
```

```
file.close()
```

Program 5: Count Number of Lines in a File

```
file = open("sample.txt", "r")
count = 0
```

```
for line in file:
    count += 1
```

```
print("Total number of lines:", count)
```

```
file.close()
```

Program 6: Count Words in a File

```
file = open("sample.txt", "r")
data = file.read()
words = data.split()
print("Number of words:", len(words))
```

```
file.close()
```

Program 7: Copy Content from One File to Another

```
file1 = open("sample.txt", "r")
data = file1.read()

file2 = open("copy.txt", "w")
file2.write(data)

file1.close()
file2.close()

print("File copied successfully")
```

Problem Statement

Problem 1

Write a Python program to:

1. Create a file named student.txt.
2. Write the names of 5 students into the file.
3. Display the contents of the file.

```
file = open("student.txt", "w")

for i in range(5):
    name = input("Enter student name: ")
    file.write(name + "\n")

file.close()

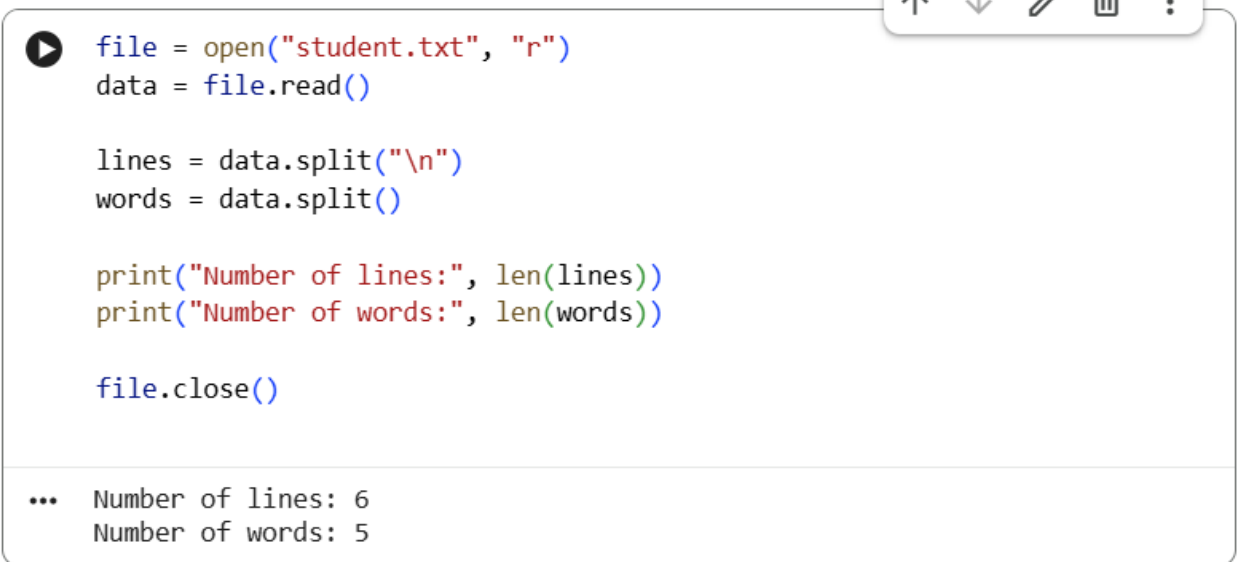
file = open("student.txt", "r")
print(file.read())
file.close()
```

```
... Enter student name: pranav
Enter student name: gajanan
Enter student name: om
Enter student name: chinmay
Enter student name: yash
pranav
gajanan
om
chinmay
yash
```

Problem 2

Write a Python program to:

1. Read the contents of a file.
2. Count the number of **lines and words** in the file.
3. Display the results.



```
▶ file = open("student.txt", "r")
  data = file.read()

  lines = data.split("\n")
  words = data.split()

  print("Number of lines:", len(lines))
  print("Number of words:", len(words))

  file.close()
```

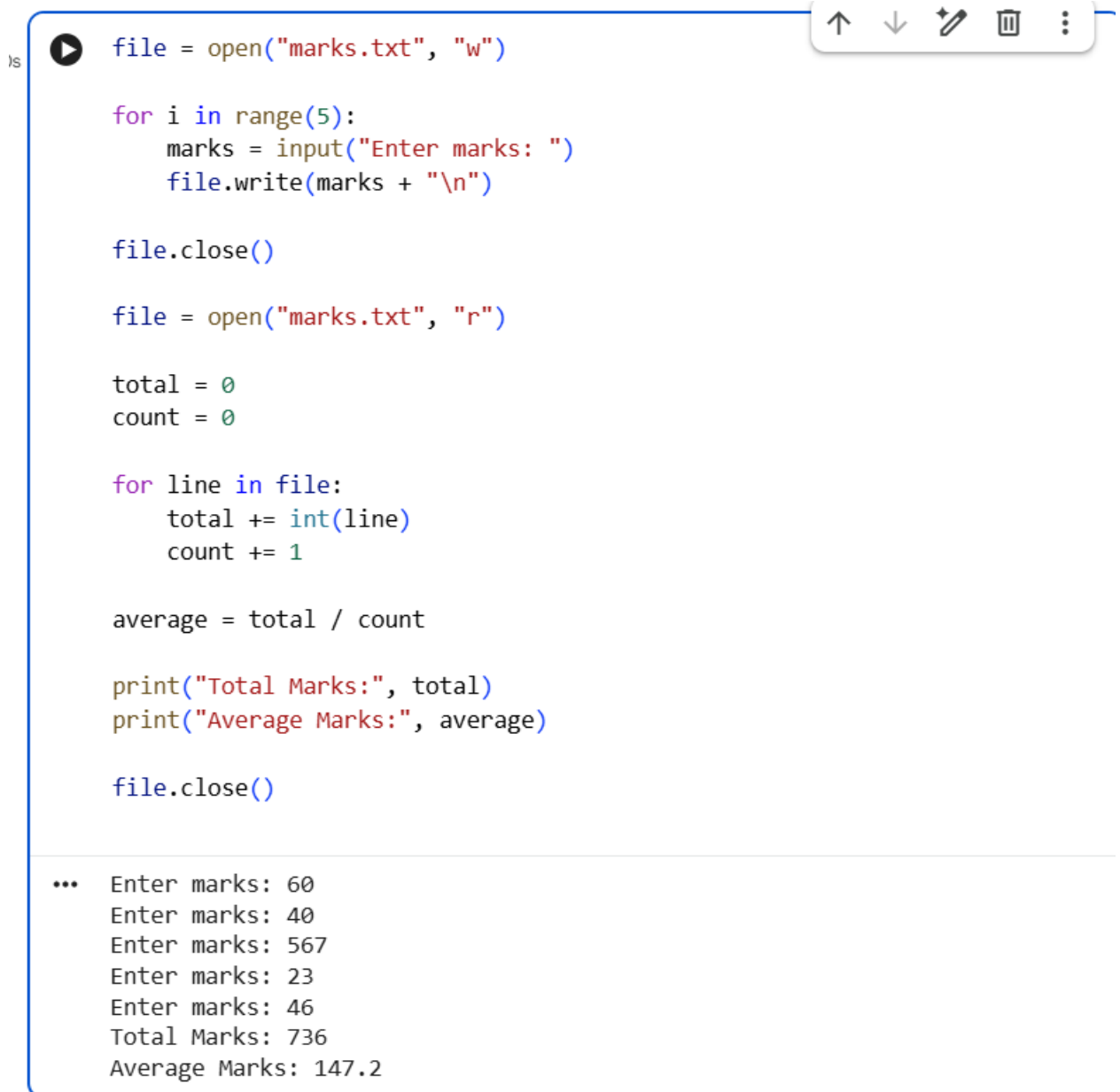
The image shows a Python IDE window with a toolbar at the top containing icons for undo, redo, run, save, and a menu. The code is written in a light blue font on a white background. The output is displayed in a separate section at the bottom of the window.

```
... Number of lines: 6
    Number of words: 5
```

Problem 3: Student Marks File

Write a Python program to:

1. Accept marks of 5 students from the user.
2. Store the marks in a file.
3. Read the marks from the file.
4. Calculate and display the **total and average marks**.



```
file = open("marks.txt", "w")

for i in range(5):
    marks = input("Enter marks: ")
    file.write(marks + "\n")

file.close()

file = open("marks.txt", "r")

total = 0
count = 0

for line in file:
    total += int(line)
    count += 1

average = total / count

print("Total Marks:", total)
print("Average Marks:", average)

file.close()
```

... Enter marks: 60
Enter marks: 40
Enter marks: 567
Enter marks: 23
Enter marks: 46
Total Marks: 736
Average Marks: 147.2

Experiment 9: Python Arrays – Programs to Study Arrays and Built-in Methods

Aim:

To study Python arrays and implement programs using built-in array methods for storing, accessing, and manipulating elements.

Objectives

- To understand arrays in Python
 - To create and initialize arrays
 - To perform operations such as insertion, deletion, and traversal
 - To use built-in methods of arrays
 - To perform searching and modification operations on arrays
-

Theory

1. Arrays in Python

An **array** is a data structure used to store multiple elements of the **same data type** in a single variable.

Python provides an **array module** to create arrays.

To use arrays, the array module must be imported.

Example:

```
import array
```

Array syntax:

```
array_name = array.array(typecode, [elements])
```

Example:

```
import array
```

```
arr = array.array('i', [10, 20, 30, 40])
```

Here, 'i' represents integer type.

2. Type Codes in Arrays

Some commonly used type codes:

Type Code	Data Type
-----------	-----------

i	Integer
---	---------

f	Float
---	-------

d	Double
---	--------

Type Code Data Type

b Signed Integer

Example:

```
arr = array.array('f', [1.5, 2.3, 4.6])
```

3. Accessing Array Elements

Array elements can be accessed using **index values** starting from 0.

Example:

```
print(arr[0])  
print(arr[2])
```

4. Built-in Methods of Arrays

Python arrays support several built-in methods.

Common methods include:

append() – Adds an element at the end of the array

insert() – Inserts an element at a specified position

remove() – Removes a specific element

pop() – Removes element from a given index

index() – Returns the index of an element

reverse() – Reverses the order of elements

Example:

```
arr.append(50)  
arr.insert(1, 15)  
arr.remove(30)
```

5. Traversing an Array

Array elements can be accessed using loops.

Example:

```
for item in arr:  
    print(item)
```

Programs

Program 1: Create and Display an Array

```
import array

arr = array.array('i', [10, 20, 30, 40])

print("Array elements:")
for item in arr:
    print(item)
```

Program 2: Access Array Elements

```
import array

arr = array.array('i', [5, 10, 15, 20])

print("First element:", arr[0])
print("Last element:", arr[-1])
```

Program 3: Insert Element into Array

```
import array

arr = array.array('i', [10, 20, 30])

arr.insert(1, 15)

print("Array after insertion:", arr)
```

Program 4: Append Element to Array

```
import array

arr = array.array('i', [10, 20, 30])
arr.append(40)
print("Array after append:", arr)
```

Program 5: Remove Element from Array

```
import array

arr = array.array('i', [10, 20, 30, 40])

arr.remove(30)

print("Array after removing element:", arr)
```

Program 6: Pop Element from Array

```
import array

arr = array.array('i', [10, 20, 30, 40])

arr.pop(2)

print("Array after pop operation:", arr)
```

Program 7: Reverse an Array

```
import array

arr = array.array('i', [1, 2, 3, 4, 5])

arr.reverse()

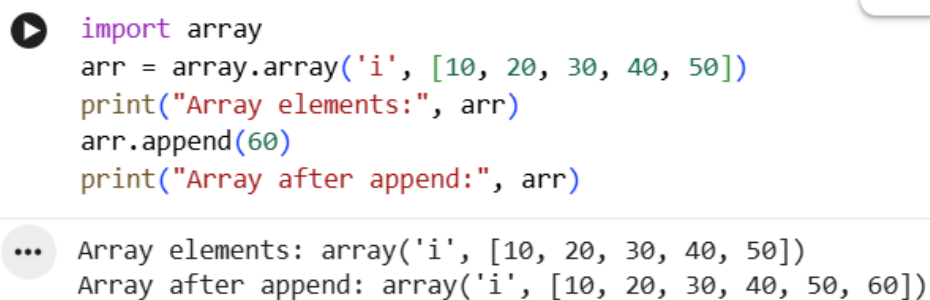
print("Reversed Array:", arr)
```

Problem Statement

Problem 1

Write a Python program to:

1. Create an integer array with 5 elements.
2. Display all elements of the array.
3. Add a new element at the end of the array using append().



```
import array
arr = array.array('i', [10, 20, 30, 40, 50])
print("Array elements:", arr)
arr.append(60)
print("Array after append:", arr)
```

Array elements: array('i', [10, 20, 30, 40, 50])
Array after append: array('i', [10, 20, 30, 40, 50, 60])

Problem 2

Write a Python program to:

1. Create an array of integers.
2. Insert a new element at a specific position.
3. Remove an element from the array.

```
import array

arr = array.array('i', [5, 10, 15, 20])

arr.insert(2, 12)
print("After insertion:", arr)

arr.remove(10)
print("After removal:", arr)
```

... After insertion: array('i', [5, 10, 12, 15, 20])
After removal: array('i', [5, 12, 15, 20])

Problem 3: Array Operations

Write a Python program to:

1. Accept 5 numbers from the user.
2. Store them in an array.
3. Display the array.
4. Find the **maximum and minimum elements**.

```
import array

arr = array.array('i')

for i in range(5):
    num = int(input("Enter number: "))
    arr.append(num)

print("Array:", arr)
print("Maximum:", max(arr))
print("Minimum:", min(arr))
```

... Enter number: 5
Enter number: 23
Enter number: 34
Enter number: 45
Enter number: 56
Array: array('i', [5, 23, 34, 45, 56])
Maximum: 56
Minimum: 5

Experiment 10: Programs to Demonstrate Data Structure Examples

Aim:

To study basic data structures in Python and implement programs demonstrating operations on lists, stacks, queues, dictionaries, and sets.

Objectives

- To understand basic data structures in Python
 - To implement list-based data structures
 - To demonstrate stack operations using lists
 - To implement queue operations
 - To perform operations on dictionaries and sets
-

Theory

1. Data Structures in Python

A **data structure** is a way of organizing and storing data so that it can be accessed and modified efficiently.

Python provides several built-in data structures such as:

- **List**
- **Tuple**
- **Set**
- **Dictionary**

These structures allow programmers to manage collections of data easily.

2. Lists

A **list** is an ordered and mutable collection of elements.

Example:

```
numbers = [10, 20, 30, 40]
```

Features:

- Ordered collection
- Allows duplicate values
- Elements can be modified

Common list operations include:

- `append()` – add element
- `remove()` – delete element

- `sort()` – sort elements
-

3. Stack

A **stack** is a linear data structure that follows the **LIFO (Last In First Out)** principle.

Operations:

- **Push** – Insert an element
- **Pop** – Remove the top element

In Python, stacks can be implemented using **lists**.

Example:

```
stack = []
stack.append(10)
stack.append(20)
stack.pop()
```

4. Queue

A **queue** is a linear data structure that follows the **FIFO (First In First Out)** principle.

Operations:

- **Enqueue** – Insert element
- **Dequeue** – Remove element

Queues can also be implemented using **lists**.

5. Dictionary

A **dictionary** stores data in **key-value pairs**.

Example:

```
student = {"name": "Alice", "age": 20}
```

Features:

- Keys must be unique
- Values can be of any data type

Common methods:

- `keys()`

- values()
 - items()
-

6. Sets

A **set** is an unordered collection of unique elements.

Example:

```
numbers = {1, 2, 3, 4}
```

Features:

- No duplicate elements
- Supports mathematical set operations

Common methods:

- add()
 - remove()
 - union()
 - intersection()
-

Programs

Program 1: List Operations

```
numbers = [10, 20, 30]
```

```
numbers.append(40)
```

```
numbers.remove(20)
```

```
print("Updated List:", numbers)
```

Program 2: Stack Implementation

```
stack = []
```

```
stack.append(10)
```

```
stack.append(20)
```

```
stack.append(30)
```

```
print("Stack:", stack)
```

```
stack.pop()
```

```
print("After pop:", stack)
```

Program 3: Queue Implementation

```
queue = []

queue.append(10)
queue.append(20)
queue.append(30)

print("Queue:", queue)

queue.pop(0)

print("After dequeue:", queue)
```

Program 4: Dictionary Operations

```
student = {"name": "John", "age": 21, "marks": 85}

print("Student Dictionary:", student)

print("Keys:", student.keys())
print("Values:", student.values())
```

Program 5: Set Operations

```
set1 = {1, 2, 3, 4}
set2 = {3, 4, 5, 6}

print("Union:", set1.union(set2))
print("Intersection:", set1.intersection(set2))
```

Program 6: Traversing a Dictionary

```
student = {"name": "Alice", "age": 20, "marks": 90}
for key, value in student.items():
    print(key, ":", value)
```

Program 7: Set Element Operations

```
numbers = {10, 20, 30}

numbers.add(40)
numbers.remove(20)

print("Updated Set:", numbers)
```

Problem Statement

Problem 1

Write a Python program to:

1. Create a list of numbers.
2. Add a new element to the list.
3. Remove an element from the list.
4. Display the updated list.

```
▶ numbers = [5, 10, 15, 20]
  numbers.append(25)
  numbers.remove(10)

  print("Updated List:", numbers)
```

```
... Updated List: [5, 15, 20, 25]
```

Problem 2

Write a Python program to:

1. Implement a **stack** using a list.
2. Perform **push and pop operations**.
3. Display the stack after each operation.

```
▶ stack = []

stack.append(10)
stack.append(20)
stack.append(30)

print("Stack after push:", stack)
stack.pop()
print("Stack after pop:", stack)
```

```
... Stack after push: [10, 20, 30]
  Stack after pop: [10, 20]
```



Problem 3: Student Data Using Dictionary

Write a Python program to:

1. Store student details (name, roll number, marks) in a dictionary.
2. Display all keys and values.
3. Update the marks of the student.

```
▶ student = {"name": "Rahul", "roll": 101, "marks": 80}
  print("Student Details:", student)
  student["marks"] = 90
  print("Updated Details:", student)

... Student Details: {'name': 'Rahul', 'roll': 101, 'marks': 80}
  Updated Details: {'name': 'Rahul', 'roll': 101, 'marks': 90}
```
